If you save this code to an HTML file (and set the API key as mentioned in the preceding note) and then open that file in your browser, you should see a bunch of photos. The key thing to note about this example is that although the web page is running from your local filesystem, it's making a request to the server at api.flickr.com. Let's walk through the code to get a better understanding of what each section is doing.

1.2.1 Setting up the request

The code starts by creating a new `XMLHttpRequest` object:

```
var xhr = new XMLHttpRequest();
if (!('withCredentials' in xhr)) {
  alert('Browser does not support CORS.');
  return;
}
xhr.open(method, url);
```

The first and last lines are the same for same-origin and cross-origin requests. The first line creates the `XMLHttpRequest` object, the last sets the HTTP method and URL.

The three middle lines highlight the difference between a same-origin-capable browser and a cross-origin-capable browser. If the browser fully supports CORS, the `XMLHttpRequest` object will contain a `withCredentials` property. You can use this property to check if the browser supports CORS. The preceding code alerts you if the browser doesn't support CORS.

1.2.2 Sending the request

Once the request is set up, you send the request to the server using the `send` method:

```
xhr.send();
```

This method initiates the HTTP request to the server. Chrome's Network tab gives you a better understanding of what the request looks like, as shown in figure 1.3. The figure shows the fact that even though the request originates from the filesystem ❶, the destination server is in fact flickr.com ❷.

Notice how the response has a header set to `Access-Control-Allow-Origin: *` ❸. The Access-Control-Allow-Origin header is the magic behind CORS. The server uses this header to indicate that cross-origin requests are allowed. The Access-Control-Allow-Origin header must always be present on a CORS response, but it's just one of many headers that can be used to configure CORS behavior. Part 2 of this book will cover these headers in greater detail.

1.2.3 Processing the response

Once the browser receives the response, it checks the response headers to verify that the cross-origin request is valid. If the request isn't valid, the browser will log an error